A Performance Comparison of the Homomorphic Encryption Schemes CKKS and TFHE

Clemens Krüger[0009−0006−2588−0069], Bhavinkumar Moriya, and Dominik Schoop[0009−0006−9971−3677]

University of Applied Sciences Esslingen, Germany

Abstract. Homomorphic encryption (HE) is a promising technique for privacy-preserving data analysis. Several HE schemes have been developed, with the CKKS and TFHE schemes being two of the most advanced. However, due to their differences, it is hard to compare their performance and suitability for a given application. We therefore conducted an empirical study of the performance of the two schemes in a comparable scenario. We benchmarked the commonly used operations addition, multiplication, division, square root, evaluation of a polynomial and a comparison function, each on a common pair of datasets with 65536 32-bit integers. Since the CKKS scheme is an approximate scheme, we set a requirement of at least 32 bits of precision to match that of the input data. Our results show that CKKS outperforms TFHE in most operations. TFHE's only advantage is its fast bootstrapping. Even though TFHE performs bootstrapping after every operation, while CKKS typically performs bootstrapping only after a certain number of multiplications, CKKS's bootstrapping still presents a bottleneck. This can be seen specifically with the comparison operation, where TFHE is much faster than CKKS in many settings, as it requires several bootstrapping operations in CKKS due to its multiplicative depth. Generally speaking, CKKS should be preferred in applications which can be parallelized. CKKS's advantages decreases in applications with a large depth that require many bootstrapping operations.

Keywords: Homomorphic Encryption · HE · Performance · CKKS · TFHE.

1 Introduction

Homomorphic encryption (HE) enables computations on encrypted data, making it possible to perform data analysis in a privacy-preserving way. In 2009, Craig Gentry proposed the first fully homomorphic encryption scheme, which was able to perform a theoretically unlimited number of additions and multiplications [12]. Over the years, many more schemes have been proposed which improved the capabilities and performance of HE. The newest and most promising of these schemes are the Cheon-Kim-Kim-Song (CKKS) and Torus Fully Homomorphic Encryption (TFHE) schemes [16]. The two schemes work in very different ways. CKKS uses real numbers as inputs, whereas TFHE uses bits or small integers.

Also, CKKS uses a batching technique which allows for parallelized operations on multiple values. These differences make it very hard to determine which scheme is most appropriate for a given application. In this paper, we perform a study to compare the performance of the two schemes for several operations executed on a common dataset.

2 Related Work

Several publications have performed various comparisons between the different FHE schemes as well as between their implementations. Jiang et. al. [15] performed an empirical study of several schemes, including BGV, BFV, CKKS and TFHE. In addition, they tested different implementations of these schemes, such as Microsoft SEAL, IBM HElib and PALISADE. However, in their benchmarks they specifically differentiate between arithmetic operations (such as addition and multiplication) and logic operations (such as AND / OR gates). Furthermore, they do not consider bootstrapping in their measurements. Such approach makes it challenging to develop selection criteria for CKKS and TFHE.

Zhu et. al. [21] compared the performance and accuracy of the CKKS scheme in the three libraries OpenFHE, HElib and Microsoft SEAL with the example of a convolutional neural network. They do not consider any other schemes for their evaluation.

Rahman et. al. [18] performed benchmarks on the three schemes BGV, BFV and CKKS in different libraries in the context of their performance on IoT devices. None of these works specifically benchmark the performance of the two schemes CKKS and TFHE in comparable circumstances, i.e. on the same datasets and using the same operations. This makes it hard to make an informed choice as to which scheme should be chosen for an application.

3 Background

In this section, we will give an overview of the two FHE schemes CKKS and TFHE, which we have chosen for this study.

3.1 CKKS

The Cheon-Kim-Kim-Song (CKKS) [8] scheme is a fully homomorphic encryption scheme designed for approximate arithmetic on real numbers. CKKS is based on the Ring Learning With Errors problem (RLWE) [19], which is a variant of the Learning With Errors problem. In LWE, secret data is represented as a set of linear equations with small errors added to them. Its security is based on the idea that it is hard to distinguish the secrets from the errors. The problem can be reduced to certain lattice problems, which are considered quantum safe. CKKS uses an encoding scheme that maps real numbers to integer coefficients of polynomials. The numbers are multiplied by a scaling factor Δ, which

dictates the precision of the numbers after the decimal point. This encoding method uses a batching technique, where multiple real numbers are encoded into one CKKS plaintext. These plaintext batches can then be encrypted, and homomorphic calculations are performed element-wise on each value (SIMD approach). CKKS itself is a leveled scheme, meaning that the maximum number of consecutive multiplications (called the multiplicative depth d) needs to be defined during setup. It does, however, offer a bootstrapping operation [7], which is able to reset the level, allowing for a theoretically unlimited number of operations. During bootstrapping the scheme's decryption mechanism is evaluated homomorphically. Consequently, it is possible to reset the multiplicative depth without needing to decrypt the ciphertext. The added error terms introduce a certain amount of noise to the ciphertexts. This noise increases with every subsequent operation, especially through multiplications.

Ciphertexts in CKKS are represented as elements of the polynomial ring $\mathbb{Z}_q[X]/(X^N + 1)$, where N is the degree of the polynomials, also called the ring dimension, and q is referred to as the ciphertext modulus. This modulus is constructed as the product of multiple large prime numbers (modulus chain), specifically $q = q_0 \cdot q_i^d$. The prime q_0 is the first modulus in this chain, and it is used during decryption. The primes q_i ($i \geq 1$) are very close in size to the scaling factor Δ. There is one of these for each level corresponding to the multiplicative depth.

To handle the noise growth during multiplication, CKKS has a special rescaling operation. After each multiplication, the ciphertext is divided by the scaling factor and one modulus q_i is dropped, effectively reducing the left-over multiplicative depth. When all levels are exhausted, only the modulus q_0 is left, which can then be used for decryption. While the scaling factor Δ controls the maximum precision after the decimal point, the difference between q_0 and Δ specifies the maximal number of bits before the decimal point.

3.2 TFHE

The Torus Fully Homomorphic Encryption (TFHE) scheme [10] is based on the RLWE problem as well as a variant called Torus LWE (TLWE). The scheme is based on the FHEW [11] scheme, which it improved by providing a very fast bootstrapping operation. TFHE's bootstrapping is also referred to as functional bootstrapping or programmable bootstrapping (PBS), because it allows to evaluate a function during bootstrapping without additional performance costs. This is achieved through lookup tables (LUTs), which can be evaluated as a byproduct of the bootstrapping. TFHE's bootstrapping procedure is so optimized, that it is executed by default after every operation.

TFHE is a hybrid scheme which combines techniques from traditional LWE constructions as well as GSW [13]. Its fundamental arithmetic and encoding occur on the torus ($\mathbb{T} = \mathbb{R}/\mathbb{Z}$), giving it its name.

Originally, TFHE ciphertexts represented individual bits, and the operations were simply binary gates such as AND. Recently, however, the scheme was expanded to support multi-bit ciphertexts up to 8 bits, which are often referred to

as small integers. From these small integers, one can then construct encryptions of larger integers (e.g. 32 bits), by splitting them into multiple ciphertexts. This can be done in two ways:

- Radix representation: A given integer is represented in a base $B \in \mathbb{N}$ (at most 4 bits integer). If a given integer $m \in \mathbb{N}$ is represented as $m = m_0 + m_1 \cdot B + m_2 \cdot B^2 + \cdots + m_k \cdot B^k$, then each m_i is encrypted independently.
- CRT (Chinese Reminder Theorem) representation: The given integer $n \in \mathbb{N}$ is represented as a product of many small coprime integers of size at most 4 bits.

The CRT representation can potentially be faster than the radix-based one, however it does not support all operations on ciphertexts. To perform operations on larger integers on the basis of multiple small integers, a carry is often needed. Therefore, the space of a small integer is divided into a message space and a carry space.

4 Experimental Setup

We compare the performance of the two schemes CKKS and TFHE for several operations. To make the results comparable, we perform our measurements in the same environment. In the following, we describe the setup of our study, including utilized hardware, software libraries and dataset, as well as which parameter sets we use and which measurements we take.

4.1 Hardware

We performed our measurements on a system with 2 Intel Xeon Gold 5315Y (3.2GHz) and 512GB RAM running Ubuntu 22.04.4. We did not utilize any multi threading other than what our chosen libraries perform out of the box.

4.2 Dataset

All operations are evaluated using a common dataset to ensure a meaningful comparison across the two schemes. The dataset consist of two randomly generated sequences A and B of 65,536 (2^{16}) unsigned 32-bit integers each. Sequence A is employed for the measurement of unary operations such as square root, while the elements of A and B serve as the operands for binary operations such as addition.

The input representation varies depending on the encryption scheme. In CKKS, numbers are scaled down to the range [0,1], with the smallest distinguishable step being $2.3283064e-10$. In contrast, TFHE operates on small integers, which can be combined to represent bigger integers (see Section 5 for more detail).

Library	CKKS	TFHE	Maintained
OpenFHE [5]	✓	✓	✓
PALISADE [3]	✓	✓	
HELib [14]	✓		✓
Microsoft SEAL [2]	✓		✓
TFHE-rs [20]		✓	✓
HEANN [1]	✓		
TFHE [4]		✓	

Table 1. Libraries that implement CKKS or TFHE

4.3 Software Libraries

There are many different software libraries that implement common FHE schemes, including CKKS and TFHE. Some are specific to certain schemes, others implement multiple schemes in one library. In this section, we list libraries that implement CKKS, TFHE or both and explain our choice of library for this study. It should be noted that many more FHE implementations exist, such as Apple's Swift HE library. However, we excluded them because they do not implement either CKKS or TFHE.

Table 1 lists the commonly used libraries for the two chosen schemes and indicates whether they are still actively maintained. The libraries OpenFHE and PALISADE are the only ones on the list that implement both schemes. PALISADE is the predecessor of OpenFHE. All its features were added to OpenFHE and it is no longer maintained. The libraries HEANN and TFHE are also abandoned. They are the original implementations of CKKS and TFHE respectively, created by the schemes' authors.

For the CKKS scheme, the actively maintained options are therefore OpenFHE, HELib and Microsoft SEAL. To make our study relevant for realistic scenarios, we require specific features with regards to CKKS, namely bootstrapping support and encrypted polynomial approximation (e.g. through Chebyshev polynomials) in addition to the basic operations of addition and multiplication. Both HELib and SEAL do not support either. Furthermore, SEAL does not appear to support necessary parameter sets, specifically a sufficiently high ring dimension. Therefore, we choose the OpenFHE library (v1.2.3) for CKKS in this study.

For the TFHE scheme, we are left with OpenFHE and TFHE-rs. In its original form, the TFHE scheme operated purely on binary values. Over time, the capability to calculate on multi-bit inputs was added. As described in Section 4.2, our benchmarks are executed on datasets containing 32-bit integers. To work with these inputs in TFHE, we need to have the ability to combine these small integers to construct bigger integers from them. The TFHE-rs library provides the option to perform the available operations on multiple integer sizes, where a bigger integer is transparently composed by multiple small integers. The OpenFHE library only has sporadic support for ciphertexts bigger than individual bits. Basic operations are implemented in the form of binary gates (AND, OR, etc.). Small integers are only supported for specific functions such as sign evaluation.

Parameter	Value
Security Level	128 bit
Ring Dimension	2^{16}
Multiplicative Depth	26
ScaleModSize	51
FirstModSize	53

Table 2. CKKS Parameter Set

There is no built-in support for larger integers such as 32 bits. Therefore, we choose the TFHE-rs library (v0.11) for the TFHE benchmarks in this study.

4.4 Parameter Choice

When it comes to the choice of parameters for any given scheme, the overall goal is to achieve a certain security level while still maintaining the best possible performance. For this study, our goal is to maintain a minimum security level of 128 bits. In contrast to TFHE, where bootstrapping is typically automatically performed after every operation, CKKS requires more nuance. Its bootstrapping is a costly operation, which is why it is usually not executed after every operation, but rather in larger intervals. While some of our tested operations by themselves would not require bootstrapping due to their small depth, our goal is to show the performance of the operations as though they were part of a larger algorithm in order to make them more representative. We therefore select a parameter set for CKKS that allows bootstrapping while allowing the required precision of 32 bit for this evaluation. In addition, we need to ensure that our parameter set satisfies at least 128 bits of security. Table 2 shows our chosen parameters set.

The security of a CKKS parameter set is mainly driven by the relationship between the ring dimension and the ciphertext modulus ($\frac{N}{q}$). The ciphertext modulus is defined as $q_0 \cdot q_i^d$, where q_0 is the FirstModSize, q_i is the ScaleModSize and d is the multiplicative depth. We therefore need to find the best trade-off between the scaling factors and the ring dimension. On the one hand, we need the scaling factors to facilitate our 32 bit precision requirement and the depth to be high enough to avoid bootstrapping too often, while on the other hand keeping the ring dimension low for performance. Our goal is to keep the ring dimension from exceeding 2^{16}, as performance typically degrades rapidly after this point. However, this requires us to set our parameters tightly, because limiting the ring dimension also requires us to limit the ciphertext modulus `Q` in order to keep 128 bits of security. The maximum possible size of the scaling factor (`ScaleModSize`) in this configuration was 51 bits, with a first modulus size (`FirstModSize`) of 54. We then set the multiplicative depth to 26, of which 12 will be consumed during bootstrapping. This leaves us with 14 multiplications between bootstrapping operations.

For TFHE, we utilize the parameter set `PARAM_MESSAGE_2_CARRY_2_KS_PBS`, which is provided by the TFHE-rs library. It defines that each small integer ciphertext consists of 2 bits of message space and 2 bits of carry space. `KS_PBS`

specifies that the so-called key switching operation is performed before bootstrapping, which is the faster option.

This parameter set utilizes a tweaked uniform noise distribution and provides a minimum of 128 bits of security with a bootstrapping failure probability fixed at $\texttt{p\_error} = 2^{-64}$.

4.5 Measurements

All measurements will be performed for unary operators such as the square root on every element of dataset A and for binary operators such as addition on every element pair (a_i, b_i) from dataset A and B respectively. In the case of CKKS, the datasets will be divided into batches and then encrypted, whereas TFHE encrypts every value separately.

Throughout the paper, we will consider the throughput, latency and precision of each operation. Since the CKKS scheme operates on batches of multiple values in each ciphertext, whereas TFHE ciphertexts contain individual values, we need to distinguish between the time it takes to calculate an operation on a (pair of) ciphertext(s), versus the amortized time for a single (pair of) value(s).

Latency is defined as the time it takes to complete an operation on a ciphertext, i.e. the latency until the result is returned after starting an operation.

Throughput on the other hand is defined here as the amortized runtime for an operation on a single value from the previously defined dataset of 32-bit integers in the representation of the respective scheme. In the CKKS scheme, this is therefore determined by dividing the latency by the batch size. In TFHE on the other hand, latency and throughput are the same, due to the lack of ciphertext batching.

The **Precision** gives for each operation the relative error of the homomorphically calculated and then decrypted results compared to the results calculated in plaintext. Specifically, the relative error for each result $r_i = Dec(Enc(a_i) \circ_{HE} Enc(b_i))$ with $a_i \in A, b_i \in B$ is calculated as $\frac{|(a_i \circ b_i) - r_i|}{max(A \circ B)}$, where $max(A \circ B)$ denotes the maximum possible result of the operation $\circ_{HE}$. We will use this measurement to determine the parameter set for the encryption, specifically for the CKKS scheme, where a trade-off between precision and performance is possible depending on the parameters. The goal is to select parameters so that the relative error does not exceed 2^{-32}, i.e. the minimum relative error throughout all measurements must not go beneath the 32-bit precision we defined for the input datasets.

To test the influence of CKKS's batching technique, we perform our CKKS measurements with two different batch sizes, namely 8192 and 1. We chose the 8192 as a representative for a higher batch size, as it is a good compromise. Higher batch sizes consume more memory, making them more impractical to use. We test this batch size in comparison with the lowest possible batch size of 1.

5 Operations

This section provides a list of the operations we chose to benchmark. For each scheme, we provide details on how the operations are implemented respectively. In the CKKS scheme, all operations are executed on multiple values at the same time, due to the batching technique of the ciphertexts. Therefore, every operation can be seen as Single Input Multiple Data (SIMD). In TFHE, all operations are based on 32-bit integers, which are constructed from several small integer ciphertexts as described in Section 3.2.

5.1 Addition

CKKS: Addition is one of the natively supported operations in the CKKS scheme. The addition is performed on two ciphertexts, each of which contain a number of encrypted values. Therefore, the addition in CKKS can be seen as element-wise addition of two vectors.

TFHE: TFHE-rs uses element-wise addition of two vectors of small integers of the form $(a_1, a_2, \cdots, a_n, b)$ (where b is the number of small integers that form a single 32 bit integer) to perform the addition of the corresponding plaintexts. If the results become larger than the 32 bit limit, they overflow.

5.2 Multiplication

CKKS: The CKKS scheme handles multiplication in a similar way as addition: When two ciphertexts are multiplied, the result corresponds to an element-wise multiplication of two vectors of numbers. However, multiplication consumes levels, because a rescaling operation is applied to reduce the growth of the plaintext magnitude.

TFHE: In TFHE-rs, multiplication of two ciphertexts follows techniques introduced in GSW [13]. It uses a flattening method (called Gadget matrix) that modifies vectors without affecting dot products.

5.3 Division

CKKS: The division operation is not natively supported in the CKKS scheme. If the divisor is known as plaintext, then the inverse of the divisor can be calculated easily and then multiplied as a plaintext constant. If the divisor is encrypted, the calculation of the inverse $1/x$ is typically approximated using techniques such as Chebyshev polynomials. With the range of the dataset for this paper being $(0, 1)$, a problem presents itself: Since $1/x$ is undefined at $x = 0$, the approximation for x near 0 is not accurate. An alternative would be to keep the original scale of $(0, 2^{32}]$. However, this would lead to very large bounds for the approximation, which significantly decreases the accuracy of the result, even with a high polynomial degree. Additionally, this requires a large ciphertext magnitude. Since the

maximum ciphertext magnitude is determined by the difference between the size of the first modulus and of the scaling factor, this either requires the reduction of the scaling factor (leading to lower precision after the decimal point) or the increase of the first modulus (possibly making it larger than typical word sizes such as 64-bit and therefore reducing performance). Ideally, one would choose an approximation interval that is both small and not close to 0 (i.e. ≥ 1), such as $(1, 2)$. In this case, the required degree for achieving high precision is quite low, as the interval is small and not close to 0. When developing an application using CKKS, one should make sure that any ciphertexts that are to be divided are scaled accordingly. In this paper, we assume an interval of $(1, 2)$.

TFHE: Division in TFHE is performed as integer division. The operation results in a separate quotient and remainder.

5.4 Square Root

CKKS: Just like the division operation, the square root is not natively supported in CKKS, and should therefore be approximated using the same techniques as the division operation. This approximation is much more straightforward however, as it is well defined in the range $(0, 1)$.

TFHE: The square root cannot be executed natively in TFHE either. It must be approximated. We do this by applying the Newton-Raphson method, which approximates the integer closest to the square root of an encrypted integer $N \in \mathbb{Z}$ using i iterations. The method can be seen in Algorithm 1.

Algorithm 1 Newton-Raphson square root approximation

Require: $N \in \mathbb{Z}$, $i \in \mathbb{N}$
Ensure: Approximated square root of N
1: $x \leftarrow \frac{N}{2}$
2: **for** $i \leftarrow 1$ **to** i **do**
3: $\quad x \leftarrow \frac{1}{2} * (x + \frac{N}{x})$
4: **end for**
5: **return** x

5.5 Polynomial Evaluation

To test a more complex calculation that includes several additions and multiplications, we choose a random polynomial $f(x) = x^5 + 5x^4 + 4x^3 + x^2 + 2x$ which we evaluate in encrypted form. In FHE, it is important to keep the multiplicative depth as low as possible, which is why we evaluate our polynomial based on the Paterson-Stockmeyer method [17], making it possible to evaluate it with a depth of $\sqrt{n}$ instead of the polynomial degree n: We first divide f into

two lower-degree polynomials $f_1(x) = x^3 + 5x^2 + 4x$ and $f_2(x) = x^2 + 2x$. We can then calculate $f(x) = x^2 * f_1(x) + f_2(x)$, resulting in a maximum depth of 3 non-scalar multiplications, instead of 5. As this calculation simply consists of a number of additions and multiplications, they can be implemented easily in both CKKS and TFHE.

5.6 Comparison Operation

CKKS: Performing comparisons in encrypted form is one of the most challenging operations in FHE, while at the same time being commonly used in many applications. The operation is not natively supported in CKKS. In addition, it is hard to approximate accurately as it is a step function and therefore not continuous. General approximation techniques such as Chebyshev polynomials are not suitable for this purpose. There has been some research into approximation techniques specific to this purpose. Cheon et.al. [9] constructed two polynomials $g(x)$ and $f(x)$ with a certain degree n which, when combined, are able to approximate the general shape of a step function. They then propose to iteratively apply $g(x)$ d_g times, followed by $f(x)$ d_f times to approach a step function with the desired precision. Their proposed algorithm for comparison is shown in Algorithm 2.

Algorithm 2 Comparison Function by Cheon et.al.

Require: $a, b \in [0, 1]$, $n, d_f, d_g \in \mathbb{N}$
Ensure: Approximated value of 1 if $a > b$, 0 if $a < b$ and 0.5 otherwise
1: $x \leftarrow a - b$
2: **for** $i \leftarrow 1$ **to** d_g **do**
3: $\quad x \leftarrow g_n(x)$
4: **end for**
5:
6: **for** $i \leftarrow 1$ **to** d_f **do**
7: $\quad x \leftarrow f_n(x)$
8: **end for**
9: **return** $(x + 1)/2$

The authors determined that the optimal combination of the parameters d_f, d_g and n is to select $n = 4$ and to set d_f and d_g as high as necessary until the required precision is achieved. For this paper, we follow this recommendation while setting the parameters high enough such that the relative error introduced by the approximation and the encryption combined does not exceed 2^{-32}.

TFHE: The comparison in TFHE is performed using a comparator circuit. Since TFHE supports logic operations, such a circuit can be evaluated in encrypted form without the need for approximation.

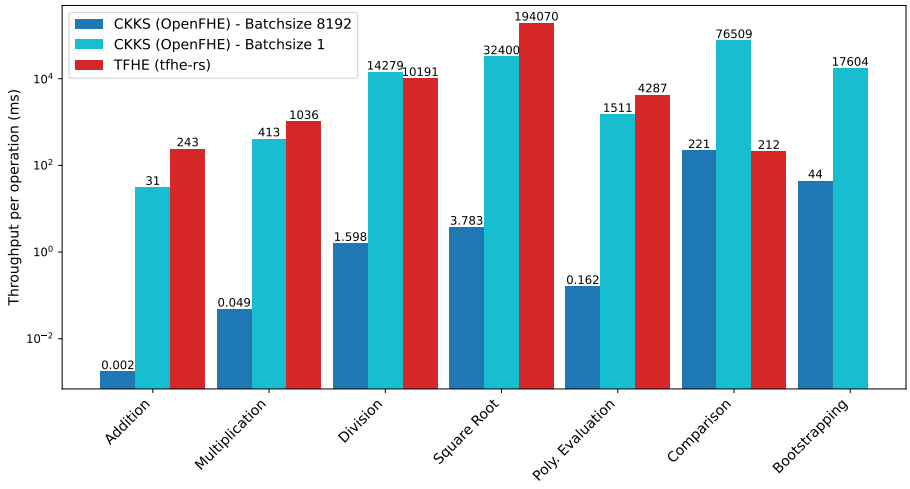

Fig. 1. Throughput of all benchmarked operations (milliseconds)

5.7 Bootstrapping

CKKS: The bootstrapping operation in CKKS resets the depth to allow for additional operations to be performed. However, bootstrapping also introduces noise to the ciphertext and therefore limits its precision. To reach our precision goal of 32 bits, we apply the iterative bootstrapping method developed by Bae et al. [6], which is implemented in the OpenFHE library.

TFHE: Since TFHE performs bootstrapping by default as part of each operation, we do not benchmark the bootstrapping operation by itself. In the case of functional bootstrapping, the calculation is performed as part of the bootstrapping itself through the use of LUTs.

6 Results

In the following, we present the throughput and latency of two measurements with different batch sizes in CKKS, as well as our TFHE measurements.

6.1 Throughput

Figure 1 shows the throughput for an individual execution of each operation. As discussed above, for CKKS the throughput corresponds to the amortized runtime defined as the runtime of a batch divided by the batch size.

It is immediately clear that CKKS has a much better throughput when using higher batch sizes throughout all measured operations.

When using a higher batch size, it can be seen that for the basic operations of addition and multiplication, the CKKS scheme shows much better throughput compared to TFHE. Here, CKKS only requires fractions of a millisecond, while TFHE is on the order of hundreds of milliseconds to a second. The division and square root calculation in CKKS is considerably slower than the addition and multiplication, due to the fact that it needs to be approximated. To achieve our chosen precision requirements, the approximation is performed using Chebyshev polynomials with very high orders, which increases the complexity. However, CKKS still performs better than TFHE here. The division specifically appears to be a hard operation in TFHE, as a single execution takes about 10 seconds. The square root is the only operation in our list which cannot be calculated directly in TFHE, but must be approximated just like in CKKS. Here, we used the Newton-Raphson method. Due to its iterative nature and the fact that it uses costly divisions, the runtime is extremely high.

For the evaluation of our polynomial, we see the same trends as with addition and multiplication. We chose this benchmark to showcase how performance scales when several basic operations are used to calculate a more complex function. The polynomial evaluation consists of three ciphertext-ciphertext multiplications, three multiplications by constant and four additions. As we can see, the two schemes exhibit similar performance scaling compared to the individual additions and multiplications.

The comparison function on the other hand shows very different results. Here, the throughput is almost identical between the two schemes. This can be attributed to the fact that approximating the comparison function is difficult due to it being a discontinuous function. As it requires a large number of multiplications for accurate results, regular bootstrapping operations must be performed throughout the approximation, to avoid excessive depth. However, the bootstrapping operation in CKKS is much slower compared to TFHE, where bootstrapping is performed as part of every single operation with the use of the aforementioned functional bootstrapping, which is why we did not benchmark it separately. As can be seen in the graph, a single bootstrapping operation with batch size 8192 takes 44 ms per slot. Throughout the comparison function, we need to perform four bootstrapping operations. These operations make up a large part of the approximation.

6.2 Latency

Figure 2 shows the latency of each tested operation, i.e. the time it takes for one operation (batch) to finish. These results are identical to the throughput in the case of CKKS with batch size 1 as well as TFHE. The only difference is for CKKS with a higher batch size. One can see that for basic addition and multiplication, CKKS still outperforms TFHE, albeit with less margin. The same goes for the polynomial evaluation. The division function on the other hand is now slightly slower compared to TFHE. When looking at the latency, the comparison function is an extreme outlier. It is now vastly outperformed by the TFHE comparison function. It can be seen that the main driver for this effect

is the performance of the bootstrapping operation. With higher batch size, this operation takes considerably longer compared to lower batch sizes. This is in contrast to the other operations in CKKS, where batch size appears to have almost no influence.

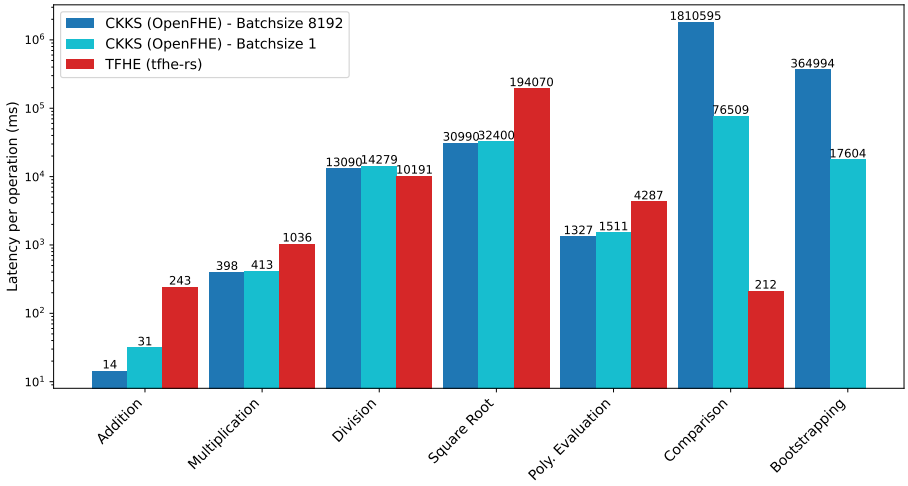

Fig. 2. Latency of all benchmarked operations (milliseconds)

7 Conclusion

Our evaluation of the schemes CKKS and TFHE show their respective strengths and trade-offs across different types of calculations. We tested the two schemes on multiple commonly used operations, each performed on a common dataset and with equal requirements for result precision. CKKS shows significant advantages when applications can leverage its batching capabilities. Lower batch sizes in CKKS do not appear to influence the latency in most calculations, except for the bootstrapping operation. The bootstrapping in CKKS seems to be the main bottleneck of performance, especially at higher batch sizes. The comparison operation is a prime example for an operation that is affected by this, as it requires multiple bootstrapping operations to avoid the depth to become excessive.

As CKKS is an approximate scheme, the precision of the results can be adjusted in order to achieve a certain trade-off between precision and runtime. In our study, we utilized a target precision of 32 bits. Increasing or decreasing this requirement would have considerable impact on the runtime. The first reason for this, is that the approximated functions for division, square root and comparison would require higher degree polynomials or more iterations to achieve higher precisions and vice versa. Additionally, different precision targets would also

affect the necessary parameters for CKKS, potentially requiring different ring dimensions, which would have an additional impact on performance.

References

1. HEANN library, https://web.archive.org/web/20230627031842/https://github.com/snucrypto/HEAAN
2. Microsoft SEAL (release 4.1), https://github.com/Microsoft/SEAL
3. PALISADE library, https://palisade-crypto.org/
4. TFHE library, https://github.com/tfhe/tfhe
5. Badawi, A.A., Bates, J., Bergamaschi, F., Cousins, D.B., Erabelli, S., Genise, N., Halevi, S., Hunt, H., Kim, A., Lee, Y., Liu, Z., Micciancio, D., Quah, I., Polyakov, Y., R.V, S., Rohloff, K., Saylor, J., Suponitsky, D., Triplett, M., Vaikuntanathan, V., Zucca, V.: OpenFHE: Open-source fully homomorphic encryption library, https://eprint.iacr.org/2022/915, published: Cryptology ePrint Archive, Paper 2022/915
6. Bae, Y., Cheon, J.H., Cho, W., Kim, J., Kim, T.: META-BTS: Bootstrapping precision beyond the limit. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. pp. 223–234. ACM (2022). https://doi.org/10.1145/3548606.3560696, https://dl.acm.org/doi/10.1145/3548606.3560696
7. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: Bootstrapping for approximate homomorphic encryption. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2018. pp. 360–384. Springer International Publishing (2018). https://doi.org/10.1007/978-3-319-78381-9_14
8. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic encryption for arithmetic of approximate numbers. In: Advances in Cryptology – ASIACRYPT 2017, vol. 10624, pp. 409–437. Springer International Publishing (2017)
9. Cheon, J.H., Kim, D., Kim, D.: Efficient homomorphic comparison methods with optimal complexity. In: Advances in Cryptology – ASIACRYPT 2020. pp. 221–256. Lecture Notes in Computer Science, Springer International Publishing (2020)
10. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: Fast fully homomorphic encryption over the torus. Journal of Cryptology **33**(1), 34–91 (2020)
11. Ducas, L., Micciancio, D.: FHEW: Bootstrapping homomorphic encryption in less than a second. In: Oswald, E., Fischlin, M. (eds.) Advances in Cryptology – EUROCRYPT 2015, vol. 9056, pp. 617–640. Springer Berlin Heidelberg (2015). https://doi.org/10.1007/978-3-662-46800-5_24, http://link.springer.com/10.1007/978-3-662-46800-5_24, series Title: Lecture Notes in Computer Science
12. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: Proceedings of the 41st Annual ACM Symposium on Theory of Computing. pp. 169–178. ACM (2009)
13. Gentry, C., Sahai, A., Waters, B.: Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In: Canetti, R., Garay, J.A. (eds.) Advances in Cryptology – CRYPTO 2013, vol. 8042, pp. 75–92. Springer Berlin Heidelberg (2023). https://doi.org/10.1007/978-3-642-40041-4_5, http://link.springer.com/10.1007/978-3-642-40041-4_5, series Title: Lecture Notes in Computer Science
14. Halevi, S., Shoup, V.: Design and implementation of HElib: a homomorphic encryption library. Cryptology ePrint Archive (2020), https://eprint.iacr.org/2020/1481
15. Jiang, L., Ju, L.: FHEBench: Benchmarking fully homomorphic encryption schemes

16. Marcolla, C., Sucasas, V., Manzano, M., Bassoli, R., Fitzek, F.H., Aaraj, N.: Survey on fully homomorphic encryption, theory, and applications. Proceedings of the IEEE **110**(10), 1572–1609 (2022), https://ieeexplore.ieee.org/abstract/document/9910347/, publisher: IEEE
17. Paterson, M.S., Stockmeyer, L.J.: On the number of nonscalar multiplications necessary to evaluate polynomials. SIAM Journal on Computing **2**(1), 60–66 (1973). https://doi.org/10.1137/0202007, http://epubs.siam.org/doi/10.1137/0202007
18. Rahman, T., Osmani, A.M.I.M., Rahman, M.S., Shibly, M.M.A., Islam, S.: Benchmarking fully homomorphic encryption libraries in IoT devices. In: Proceedings of the 11th International Conference on Networking, Systems, and Security. pp. 16–23. ACM (2024). https://doi.org/10.1145/3704522.3704546, https://dl.acm.org/doi/10.1145/3704522.3704546
19. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. Journal of the ACM **56**(6), 34:1–34:40 (2009)
20. Zama: TFHE-rs: A pure rust implementation of the TFHE scheme for boolean and integer arithmetics over encrypted data
21. Zhu, H., Suzuki, T., Yamana, H.: Performance comparison of homomorphic encrypted convolutional neural network inference among HElib, microsoft SEAL and OpenFHE. In: 2023 IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE). pp. 1–7. IEEE (2023)